Listing 6.3 networks.css

```
.grid {
      stroke: black;
      stroke-width: 1px;
      fill: red;
}
.arc {
       stroke: black;
       fill: none;
}
.node {
      fill: lightgray;
      stroke: black;
      stroke-width: 1px;
}
circle.active {
            fill: red;
}
path.active {
            stroke: red;
}
```

6.1.2 Adjacency matrix

As you see more and more networks represented graphically, it seems like the only way to represent a network is with a circle or square that represents the node and a line (whether straight or curvy) that represents the edge. It may surprise you that one of the most effective network visualizations has no connecting lines at all. Instead, the *adjacency matrix* uses a grid to represent connections between nodes.

The principle of an adjacency matrix is simple: you place the nodes along the x-axis and then place the same nodes along the y-axis. If two nodes are connected, then the corresponding grid square is filled; otherwise, it's left blank. In our case, because it's a directed network, the nodes along the y-axis are considered the source and the nodes along the x-axis are considered the target, as you'll see in a few pages. Because our network is also weighted, we'll use saturation to indicate weight, with lighter colors indicating a weaker connection and darker colors indicating a stronger connection.

The only problem with building an adjacency matrix in D3 is that it doesn't have an existing layout, which means you have to build it by hand like we did with the bar chart, scatterplot, and boxplot. Mike Bostock has an impressive example at http://bost.ocks.org/mike/miserables/, but you can make something that's functional without too much code, which we'll do with the function in listing 6.4. In doing so, though, we need to process the two JSON arrays that are created from our CSVs and format the data so that it's easy to work with. This is close to writing our own layout, something we'll do in chapter 10, and a good idea generally.